

Load Test Report: Test 3 of 3, 300 Concurrent Users

Date: 26 Sep 2026, 10:25 – 10:32 IST
Target: `https://api.iverto.ai/devhoste1`, Hostel **v2** API (`/v2/tenants/acme/...`)
Tool: k6 v2.2.0, `k6/v2/users.js`, run from one Windows laptop (one public IP)
Traffic: reads only. No passes, decisions or other writes were sent.

1. Verdict

FAIL. The app is unusable at 300 users. The server stayed up, but opening the app took **over a minute**.

SLO	Target	Result	
HTTP error rate	< 1%	0.58% (63 / 10,740)	✓ (misleading, see §4)
Checks passed	> 99%	99.41%	✓
Latency p95	< 800 ms	13.88 s (17× over)	X
Latency p99	< 2 s	15.82 s	X
Median latency		7.67 s	X
One app-open (journey), p50 / p95	~10 s	82 s / 103 s	X
GET <code>/dashboard</code> success	100%	0% (55 × 500, 2 × timeout)	X
GET <code>/roster</code> success	100%	0% (3 × 30 s timeout)	X
GET <code>/groups</code> success	100%	0% (3 × 500)	X

2. Test setup

Item	Value
Model	Closed: each virtual user opens the app (one role journey), idles 3–8 s, repeats
Profile	0 → 300 users over 1 min · hold 300 for 4 min · ramp down 30 s (total 5 m 59 s)
Role mix	student 80% · guardian 15% · warden 4% · admin 1%
Token pool	35 tokens (24 / 6 / 3 / 2), logged in 10:21–10:25, 0 × 429
Request timeout	30 s

3. Results

3.1 Totals across all three tests

Metric	30 users	100 users	300 users
Throughput (avg)	23.8 req/s	27.2 req/s	29.9 req/s
Journeys per second	2.65	3.03	3.03
Latency p50	118 ms	2.01 s	7.67 s

Metric	30 users	100 users	300 users
Latency p95	636 ms	5.78 s	13.88 s
Latency p99	1.26 s	8.63 s	15.82 s
Journey p50 / p95	9.2 s / 14.1 s	27.5 s / 47.1 s	82 s / 103 s
Failed requests	29 (0.36%)	64 (0.69%)	63 (0.58%)
Journeys cut off at end of test	0	0	145 (still waiting on the server)

10× the users of Test 1, the same ~30 req/s served. Throughput has been flat since ~30 users. Every extra user only adds waiting time. Little's law checks out: 300 users ÷ 3.03 journeys/s ≈ 99 s per cycle, which matches the measured ~82 s journey plus ~5 s think time.

3.2 Timeline (15 s buckets)

Time	Users	req/s	p50	p95	Errors
0–15 s	ramp 0 → 75	32.7	0.8 s	2.0 s	0%
15–45 s	75 → 225	24 – 37	4.0 – 4.6 s	4.9 – 5.3 s	0.4 – 1.1%
45–90 s	225 → 300	34 – 40	6.3 – 7.3 s	8.0 – 8.3 s	0.2 – 0.4%
90–150 s	300	21 – 38	7.4 – 12.7 s	8.7 – 14.1 s	0.5 – 1.6%
195–210 s	300	17.5	14.1 s	15.9 s	1.9%
210–300 s	300	19 – 37	7.0 – 11.8 s	9.0 – 15.9 s	0.2 – 0.7%
300–315 s	300	24.4	13.9 s	15.4 s	1.1%
315 s → end	ramp down	42 → 5	7.5 → 1.5 s	11.0 → 3.6 s	0.2 – 1.5%

The peak was ~40 req/s for single 15 s windows. The sustained rate never went above ~30.

3.3 Per endpoint (sorted by p95)

Endpoint	Calls	p50	p95	Max	Errors
GET /roster	3	30 s	30 s	30 s	3 × timeout (100%)
GET /dashboard	57	22.5 s	28.3 s	30 s	55 × 500 + 2 × timeout (100%)
GET /groups	3	21.3 s	24.8 s	24.8 s	3 × 500 (100%)
GET /memberships	2	15.3 s	15.3 s		0
GET /me/children	185	9.0 s	15.3 s	17.1 s	0
GET /me/children/:id	60	8.9 s	15.1 s	16.5 s	0
GET /me/student	985	8.1 s	14.6 s	16.6 s	0
GET /me/curfew	925	8.3 s	14.3 s	16.4 s	0
GET /sites	1,046	7.7 s	13.7 s	15.9 s	0
GET /me/branding	1,231	7.7 s	13.6 s	15.9 s	0
GET /me/permissions	1,349	7.7 s	13.5 s	15.9 s	0
GET /app-config	1,227	7.2 s	13.4 s	15.9 s	0
GET /me/notifications/unread-count	1,218	7.3 s	13.1 s	15.9 s	0

The spread between endpoints has almost disappeared. The cheapest (`/app-config`, 7.2 s) and the student profile (`/me/student`, 8.1 s) are nearly the same. **Time is spent waiting in a queue, not running queries.** Student requests never reached the ~15–20 s server timeout, so they returned `200` very late instead of `500`.

4. What went wrong

1. **The app is unusable.** A student opening the app waits **82 s (median) to 103 s (p95)** for the home screen's requests to finish. Real users force-close long before that, so the actual failure rate is far higher than the 0.58% HTTP errors suggest.
2. **Throughput is capped at ~30 req/s** and doesn't grow with users: 30 → 100 → 300 users gave 24 → 27 → 30 req/s.
3. **Every staff screen fails 100%.** Dashboard, roster and groups return `500` or time out. Wardens cannot see who is inside or approve anything at this load.
4. **All endpoints queue on one shared bottleneck.** `/app-config` (a constant) has the same ~7 s median as DB-heavy calls. This is the signature of an exhausted DB connection pool (or a single worker) where requests wait for a slot.
5. **Work was left unfinished when the test ended.** 145 journeys were still waiting on the server after the 30 s graceful ramp-down.
6. **Good news:** unlike the 24 Sep arrival-rate test (30 journeys/s, Caddy `502`, 2.5 min recovery), the server **did not crash**. A health check straight after the run answered in 0.4 s. Closed-model users slow down when the server slows down, so the backlog never grew enough to kill the process. A real crowd that retries and reopens the app behaves more like the 24 Sep test.

5. Fixes

Same root causes as Tests 1 and 2, now with the full picture. Order matters.

#	Problem	Fix	Expected effect
1	Unknown exact bottleneck	Pull server metrics for 10:25–10:32 IST: DB pool <code>waiting</code> count, active connections, Postgres <code>pg_stat_activity</code> , CPU, event-loop lag. Confirm pool exhaustion.	Confirms the diagnosis
2	Staff reports hold connections for 15–30 s	Dashboard: one aggregate query plus a 15–30 s cache. Roster: SQL pagination plus indexes. Groups: remove <code>N+1</code> . <code>statement_timeout</code> 5 s. Separate small pool for reports.	Frees the pool; removes all <code>500</code> s
3	Student reads slow at the source (<code>/me/student</code> 0.4 s at 30 users)	Indexes <code>(tenant_id, student_id)</code> , <code>(tenant_id, site_id)</code> , <code>(tenant_id, status, created_at)</code> , <code>(student_id)</code> on guardian links; remove guardian <code>N+1</code> in <code>/me/student</code> and <code>/me/children*</code>	~5–10× more req/s per connection
4	Static data hits the DB on every app open	Cache <code>/app-config</code> , <code>/me/branding</code> , <code>/sites</code> , <code>/categories</code> (Redis/in-memory + ETag / Cache-Control)	~35% fewer DB requests
5	Unbounded queueing	Pool size 20–30, acquire timeout 1–2 s, fast <code>503</code> + <code>Retry-After</code> on saturation. App: show "busy, retrying" and back off instead of spinning.	Fails fast instead of 80 s waits
6	One instance	≥ 2 app instances behind Caddy with active health checks; scale DB (or add a read replica for reports)	Linear headroom
7	Verify	Re-run <code>k6 run -e USERS=300 k6/v2/users.js</code> . Pass: <code>p95 < 800</code> ms, errors < 1%, staff endpoints 100% <code>2xx</code> , throughput ≥ 100 req/s.	

6. Raw files

load-test/results/2026-09-26/users-300.{csv,json,log}, users-300.csv.analysis.json, pool-300.log.

Reproduce: `node scripts/login-pool-v2.mjs && k6 run -e USERS=300 -e HOLD=4m k6/v2/users.js`